# Technical Report

Third semester final project -

AIDS (Automatic Integrated Defense System)

Supervising teachers: Caspar Facius, Jonas Wehding, Kaj Norman Nielsen
Page count:31855
Character count:22

Corad Dumitru Teofil ITEK-E21a

# Table of contents

# Summary

The project chosen and developed will replace common home defense and security measures whilst also cutting the cost on the long run. Usually trained agents are deployed to a location in order to guard it. AIDS(Automatic Integrated Defense System) will be able to replace the need of an agent, being more efficient and cheaper. The product resembles a “sentry gun” being mounted on a tripod.The main part of the product is the Gun to whom everything is either connected, bolted or ziptied, the gun, for testing purposes is an electric airsoft rifle. Having, in future iterations of the product, a high ammunition diversity starting from plastic BBs, going to rubber balls and all the way to compressed pepper balls. For this prototype the chosen type of ammunition is BBs. AIDS features an object recognition camera that can lock onto targets and shoot them from afar. The product in its current state of development and prototyping can only rotate on the X axis having a field of view of ~200º. The product is capable of both fully automatic fire and semi automatic fire although each configuration must be manually preset before deploying the product, future iterations will have this feature controlled remotely.

# Introduction

As a part of the Business Academy Aarhus Curriculum each student must choose and develop a personal project. That project must be a product of the students gained knowledge in embedded systems, electronic systems and programming gained in the past 3 months. This project simulated working in a company by giving students a deadline and specific requirements. It was developed during the course of 30 days and contains personally procured hardware; Business Academy Aarhus also provided the necessary hardware and tools to complete this project. This report will give a detailed overview of the creative process, development and prototyping whilst also including the testing and the changes made to the project during the initial development and planning.

This project was sponsored by Business Academy Aarhus.

# Case description

Each student had the possibility to develop and create their own project that would solve a current active problem.

The scope of this project is to offer clients a reliable alternative to regular home protection. This, in theory will not only aid the common citizen but also safely keep any building either it being corporate, military or just a regular shed. The product features a mounted automatic weapon that is placed on a Dynamic Tripod able to rotate and target nearby individuals. The product should be able to detect a break in, locate the target and neutralize it whilst sending a full report on a dedicated secure server.

With the current evolution in technology burglars are becoming smarter and smarter, being able to by-pass traditional home security methods, by not triggering the alarm or by making it out before the police arrives (average time – 18 minutes) and even evading on-field personnel with enough expertise due to lack of professional training. Deploying a security guard on-field is expensive and requires intensive training in order to pose a threat to burglars. This method is expensive and time consuming.

AIDS would cost less to manufacture and deploy than a fully trained, full-time security guard and would be more efficient and more intimidating. Being able to repel attackers and threats straight from the factory.[1]

## Requirements

Since the product only needs a prototype for the given purpose the requirements have been stripped down to the bare minimum. This decision was also made in order to facilitate its development in the given time. The simplified requirements are as follows:

- The product must be able to turn on the X axis (min 180º)
- The product must have a mounted camera
- The product must be able to recognise an image/person
- The product must be able to follow an image/person by centering it
- The product must be able to fire the gun

Additional non-mandatory requirements were created in order to facilitate the users handling of the product, these are optional as they would take an extended amount of time to implement and they do not change the functionality of the product:

- The product should be powered by an on-board battery
- The product should be accessible remotely
- The product should upload data to a database after every encounter
- The product should have a dedicated monitoring website
- The product should be able to turn on the Y axis (min 90º)

# Tools and hardware

## Tools

A plethora of tools were used to complete this project ranging from physical tools and electronic devices to software and libraries. Business Academy Aarhus provided all of the indeed physical tools to build this product (pliers, wire strippers, screwdrivers, screws, electrical tape, cables and connectors) as well as the electronic devices needed for testing various parts of the product (power supplies, oscilloscopes, soldering irons, a CNC drill and voltmeters). The connection to the main board was done using the provided monitor and keyboard.

In terms of software the Raspberry Pi Imager app was used intensely at the beginning of the project. The programming of the product was done in the Terminal window using the Python 3.9 programming language. This was used together with the Python libraries needed to utilize the camera and the servos.

## Hardware components

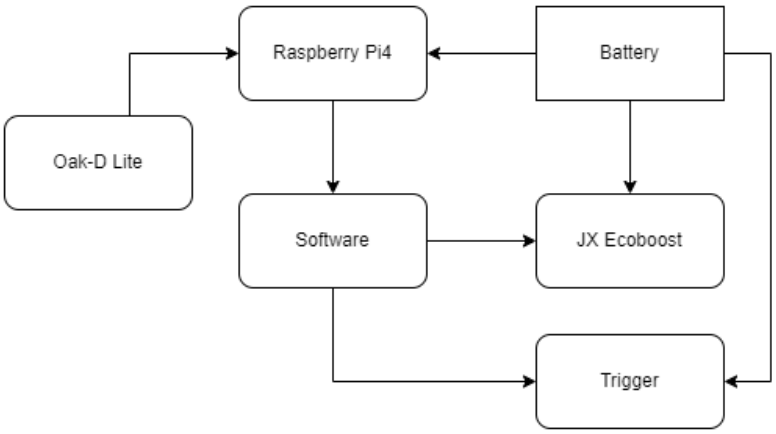
The product is composed of multiple parts, those being the main board (Raspberry Pi 4), a camera for recognition (OAK-D Lite), a servo to turn the main weapon (JX Servo EcoBoost), the main weapon (AK-47 Beta Spetsnaz) with the included battery, the tripod that holds everything (Velbon DF-51) and a veroboard with electrical components.

# The product

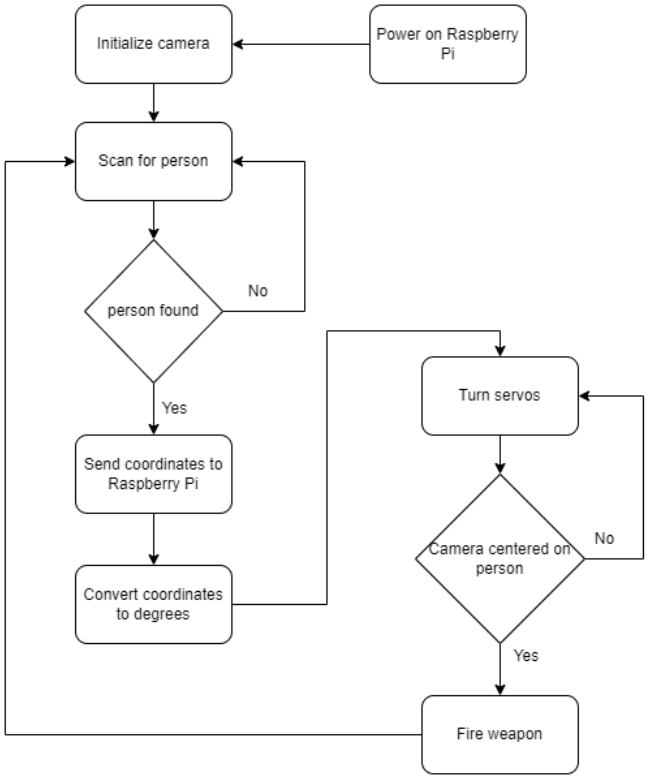
The developed product is a home defense sentry gun. It is easy to mount as all it needs is a 220V outlet for the Raspberry Pi4. The adjustable legs can help the main weapon keep a straight angle on an inclined surface. It can also be raised to the desired height for optimal functionality. It features a main gun firing 0.20 g BBs with a 30 BB swappable magazine and a ~200º turn radius so it can be placed in a corner or in a doorway. It features a rechargeable battery attached to one of the legs. The mounted camera has a range of 20 meters for detection and the high power of the main gun, 1.4 Joules (110 m/s) make it suitable for indoor and outdoor surveillance and protection.

# Block Diagrams

## Hardware block diagram




## Software flowchart



# Design

The product will be featuring the main weapon placed upside down on a stand. This stand will have servos that would turn the main weapon, one on the X axis and one on the Y axis. The battery included with the weapon will be used to power the sentry gun. A Raspberry Pi will be used to control the main weapon and the servos. One additional servo will be used to pull the trigger. A camera will be mounted on top of the gun to make the object recognition possible.

# Implementation

## Hardware

The first iteration of the AIDS system featured a twin RX-28 Dynamixel servo mount capable of moving the X axis as well as the Y axis(Figure 1.1.1). The servos were mounted on a 3D printed mount. The mount made it easier to mount the main weapon along with  the camera and the Raspberry Pi4. The RX+28 Dynamixel servos need an RS-485 signal. The UART (Universal Asynchronous Receiver Transmitter) signal had to be converted to RS-485, this was done using a RS-485 Breakout board (Figure 1.1.2). The board failed to convert the signal; the idea had to be dropped after it took too much time from the project.

Figure 1.1.1

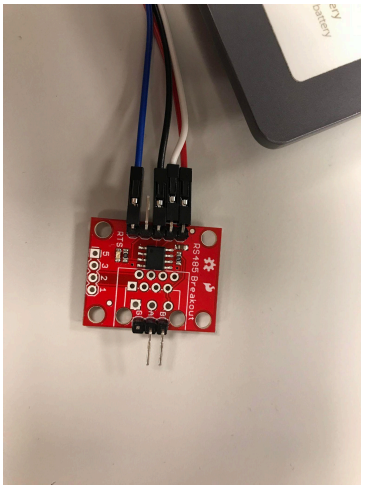


Figure 1.1.2

The movement of the main weapon was now done by a single JX EcoBoost servo mounted to a Velbon DF-51 camera stand. For the given purpose the servo was mounted using electrical tape and personally provided zip-ties. Although this might seem like the servo had a rather flimsy mount it was proven otherwise as the high amount of zip-ties and pressure points made the servo immovable.The only impediment created by using so many zip-ties is that should the servo be damaged or destroyed it would not be easy to replace, this can be easily be fixed by attaching a 3D printed or CNC-ed mount made of aluminum in further iterations of the sentry gun.

Once the servo was securely attached to the camera stand a two-piece “arm” created from carbon fiber bars and bearing was attached. One end to the servo motor and one to the handling stick of the camera mount in order to maximize the turn radius. This was done by using a CNC drill. After that the unhinged carbon fiber arm was attached to the camera handle using a regular screw. The hardware part of turning the main weapon was completed.(Figure 1.1.3)

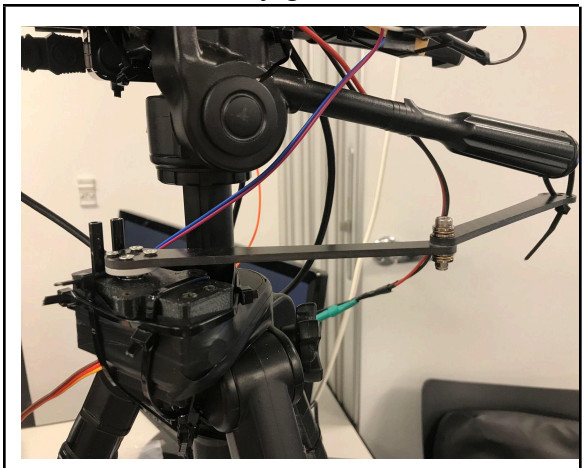


Figure 1.1.3

The main weapon was attached on top of the camera mount backwards in order to facilitate the feeding of the magazine into the weapon. This was also done in order to stabilize and balance the weapon as its top side was flat enough and had a big enough area to be placed on top. This also helped align the center of mass of the main weapon with the top of the camera mount. Once again since this is only a prototype the main weapon was secured in place using zip-ties placed around the camera mount and then onto the weapon. It was done this way in order to maximize the amount of points that the zip-ties could be attached to as neither the camera mount nor the weapon had available mount points. One small drawback of this design is that one of the zip-ties created a pressure point onto the magazine, this is not an impediment to performing the reload but it did make it more inconvenient. Future iterations of the sentry gun will feature either 3D printed mounts to encase the weapon or pre-drilled holes in the weapon and mount so it can be screwed in. This approach was not taken during the development of the weapon in order to not damage the internal motor and render the weapon unusable. (Figure 1.1.4)

Figure 1.1.4

The weapon had to be stripped down of every aesthetic or non essential part in order to reduce weight and facilitate the servos turning. After disassembling the weapon it was tested separately in order to check its functionality. The camera went through different positions on the camera mount before it reached its final position. Firstly a mount was created for the camera from a sheet of aluminum with pre-drilled holes. (Figure 1.1.5)

Figure 1.1.5

The camera mount and the camera were placed on one of the legs as the first iteration of the design featured a static camera with the main weapon moving but this was scrapped due to the low field of view of the camera thus highly restricting the whole field of view of the sentry turret. In the next step the barrel mount of the weapon was removed and a hole was drilled in it.

This facilitated the mounting of the camera in the under barrel of the weapon by using a screw. This approach was taken in order to give the camera a more solid mount to the weapon making the camera almost immovable. A possible problem that this type

of mounting could create is that, since the weapon vibrates a lot when engaged, the camera could experience “turbulences” and be unable to detect the target for a short amount of time, but since the weapon only fires in short bursts it will be able to stabilize back to normal. The vibrations could also damage the camera, an attempt was made at creating at creating a suspension system from a repurposed spring taken from the main weapon(Figure 1.1.6) but the idea was scrapped as it was ineffective with the provided tools and equipment, this could eradicate the risk of damaging the weapon but at its current state it was rendered obsolete.(Figure 1.1.7)

Figure 1.1.7

Figure 1.1.6

The main processing unit of the sentry turret, a personally provided Raspberry Pi4 was mounted to the side of the weapon as it constituted the only flat part of the turret with a big enough area to host the Raspberry Pi4. The board had its own protective case that featured two mounting holes at the back. This was beneficial to the development of the project as no special mount had to be created to mount the board to the sentry turret. The case was held by zip-ties as well providing a sturdy mount to the main weapon. All of the connections from the Raspberry Pi4 were made long enough so they would not be an impediment to the turning radius of the weapon and there was no risk of them being damaged or damaging the pins connected to the board. The camera connection posed a challenge as the only available way to connect the camera to the board was using a USB to USB-C cable that had a length of 1 meter and since the sentry turret only had a height of ~70 centimeters the cable had to be zip-tied around the legs of the camera mount.(Figure 1.1.8)

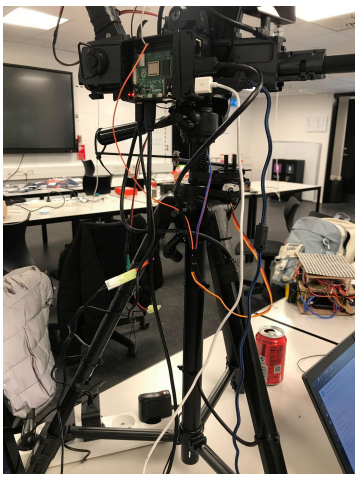


Figure 1.1.8

The battery was placed on one of the legs due to it being very slim and long. It was firstly held in place by electrical tape but after that zip-ties were added in order to provide more support to the battery and to prevent it from sliding down.(Figure 1.1.8)

FInally a vero board was designed to connect all of the electronic components and attached to the side opposite of the one that had the Raspberry Pi4 attached to it, this was done using zip-ties. The copper strips of the vero board were touching the metal body of the main weapon so a rubber band was placed securely between the vero boards and the main weapons body. This vero board is composed of two smaller vero boards held together by a screw. One board featured the relay needed to power the main weapon and the other vero board featured the power distribution. The first vero board developed was the adjacent veroboard. It had a ALQ305 relay, this was used due to the fact that the main weapon draws 10 Amps, this will burn the Raspberry Pi4 hence why the relay was chosen. It will allow high current to pass through whilst the actual switch is separated and only takes 5V to turn on. A 1N4007 rectifier diode was placed at each end of the relay to prevent stocked electricity from passing through by sending it back to the battery. After that a BC547 silicone transistor was placed acting as a secondary switch being responsible for turning “on” and “off” the coil of the relay, followed by two resistors that add up to 330 Ohms. Additional connectors were added in order to facilitate connection to the relay. (Figure 1.1.9)

```
B min = 110, Vbe = 0.7V, Ic = 10 Amps, Vcc = 5V
Vr = Vcc - Vbe
Vr = 5 - 0.7 = 4.3V

Ib = B/Ic
Ib = 110/9.4 mAmp = 11.7 mAmp

Rb = Vr/Ib
Rb = 4.3V/0.011Amp = 390 Ohm
```

Figure 1.1.9

The main board had the main power from the battery connected to it. In order to do this the cables from the main weapon had to be cut in order to bypass the trigger. A voltage regulator was added to the vero board to change the voltage from the battery which was 9.4 V to a voltage capable of powering the Raspberry Pi4 but not burn or damage it, that being 5V. This was done by soldering two 4700 uF capacitors to the vero board with a L7800 series positive voltage regulator. Following the ground of the battery was soldered to the relay. Then the motor of the main weapon had to be wired.(Figure 1.1.10)

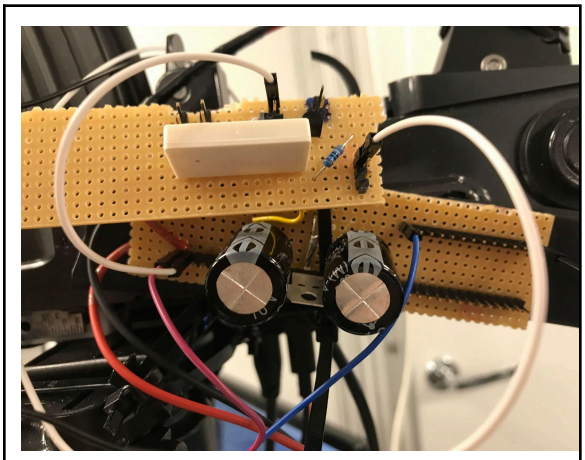


Figure 1.1.10

The weapon is powered by a high power DC motor that requires 10 Amps and 9V to run so in order to sustain such high voltage thicker cables were used. The first design of the weapon featured an additional servo used to pull the trigger but this idea was scrapped due to it making the product unnecessary mechanically complex, needing an additional “arm” to operate the trigger and then additional calibration to calculate the indeed turn radius to operate it.

A simpler option was to use the previously mentioned relay and wiring the motor straight to the vero board. At first an attempt was made to create new solder points but it was decided against due to the fact that the old connections to the trigger could be cut and repurposed. The motor wires had to be lengthened in order to accommodate the

position of the vero boards. Additional writing was connected to the pins soldered onto the vero board for ease of access.

This concluded the development and construction of the sentry turret additional zip-ties and electrical tape were added during the test of the software where indeed in order to limit the number of moving parts in the product, making it less likely to fail during testing and while operating it.

## Software

The software part began by testing each individual component separately, further on compiling all the codes into one fully functional code. Multiple Python libraries were used to achieve the given goal as well as adjacent files that facilitated the production of the product.

The software part needed to operate on an Operating System. At first the Linux Ubuntu 22.04 LTS(Long Term Support) server was chosen as an operating system due to the fact that the Apache2 web server could be installed and configured so the sentry gun could be monitored from the outside and the camera output would be placed in the web server. This idea was scrapped due to its high complexity for the current purpose of the project. It this although possible to implement this into the product in its future iterations, the only reason it was skipped during prototyping is because of the time limitations. After that the Raspberry Bullseye Debian software represented the best option for this project due to it having support for the OAK-D camera software. The desktop version of the Debian Bullseye software was chosen so that the camera output could be displayed in real time on screen for testing and further refining the software. In order to install and configure the Raspberry Pi4 Operating System the Raspberry Imager software was used on an adjacent computer.

In order for the product to work as intended multiple files had to be downloaded. Mainly the files required for the OAK-D camera. The camera had its own dedicated libraries and software with very limited support and compatibility with other libraries. (Figure 1.2.1)[2]

```
from pathlib import Path
import sys
import os
import cv2
import depthai as dai
import numpy as np
import time



from threading import Thread
from queue import Queue, Empty
from collections import deque
import subprocess


# State-machine defines the reli
from enum import Enum
class STATE(Enum):
    INIT = 0
    FOLLOWING = 1
#    RELIC_PRESENT = 2
#    AWAIT_JUDGMENT = 3
#    AWAIT_REMOVAL = 4


# Servo Stuff
import gpiozero
import numpy as np

import RPi.GPIO as GPIO
```

Figure 1.2.1

This posed a challenge for the progress of the project. FInally the spatial detection code was chosen from the main Luxonis GitHub page as it already had the facial recognition and calculation of the persons coordinates on the person in front of the camera. The given code also had a noise map integrated into it which would aid with the capturing of the persons frame. The code starts by importing all of the necessary files, libraries and repositories to operate. Then the code from the DepthAI repository is modified and implemented. The way the code works is that the code creates the possible recognition patterns using a specifically depthai folder. After that a pipeline is created to initialize and receive data from the camera.

After that the parameters of the window in which the camera will be displayed are set. Finally a connection is done between the pipeline and the window in order to have a video output. After that the main "*while True:*" function is created which sets the parameters of the video output. The bounding box is now created, this box will encase the person present in the camera into a specific area that can be calculated. Later on the  data from the boxed is used to calculate the coordinates of the person. At the end the video preview is toggled on for development and testing purposes but will be turned off in the final product as there is no need for it to feature a screen. (Figure 1.2.2)

```
# Connect to device and start pipeline
with dai.Device(pipeline) as device:

    # Output queues will be used to get the rgb frames and nn data from the outputs defined above
    previewQueue = device.getOutputQueue(name="rgb", maxSize=4, blocking=False)
    detectionNNQueue = device.getOutputQueue(name="detections", maxSize=4, blocking=False)
    depthQueue = device.getOutputQueue(name="depth", maxSize=4, blocking=False)

    startTime = time.monotonic()
    counter = 0
    fps = 0
    color = (255, 255, 255)



    while True:

        inPreview = previewQueue.get()
        inDet = detectionNNQueue.get()
        depth = depthQueue.get()

        counter+=1
        current_time = time.monotonic()
        if (current_time - startTime) > 1 :
            fps = counter / (current_time - startTime)
            counter = 0
            startTime = current_time

        frame = inPreview.getCvFrame()
```

Figure 1.2.2

Following, a small state machine was implemented into the code to switch modes from the initialisation phase to the “FOLLOWING” phase which would track movement. (Figure 1.2.1)After this a filter was created this filter smooths a 1D sequence of noisy values, at the expense of being less responsive to sudden changes in value. Once the buffer is full it simply removes the top and bottom two values and averages the remaining 6. This is effectively an outlier-rejecting average of the trailing 10 values. Further on the pins are configured in order to use the servo and firing system.(Figure 1.2.3)

```
class FilteredValue:
    """
    Smooths/filters a 1D sequence of noisy values, at the expense of being less
    responsive to sudden changes in values.  Once the buffer is full, it simply
    removes the top and bottom two values and averages the remaining 6.  This
    is effectively an outlier-rejecting average of the trailing 10 values.

    Simply call update() with new values as they come in, call read() to get
    the filtered value as of the last update.
    """
    def __init__(self, dsize=10):
        """
        Tried to make dsize configurable, but I had too many corner cases and
        ended hardcoding logic for dsize=10.  Whoops
        """
        self.hist = deque()
        #self.dsize = dsize
        self.dsize = 10
        self.curr_val = None

    def update(self, val):
        self.hist.append(val)
        if len(self.hist) > self.dsize:
            self.hist.popleft()

        # Remove any empty values
        dsort = sorted([v for v in self.hist if v is not None])

        # Use order statistics on remaining to filter extreme vals, avg the rest
        if len(dsort) == 0:
            self.curr_val = None
        elif len(dsort) in range(1, 3):
            self.curr_val = np.average(dsort)
        elif len(dsort) in range(3, 6):
            self.curr_val = np.average(dsort[1:-1])
        elif len(dsort) >= 6:
            self.curr_val = np.average(dsort[2:-2])

        return self.curr_val


    def read(self):
        return self.curr_val
```

Figure 1.2.3

The servo was configured using the “*GPIO.setup*” function. A few hardcodes were implemented in order to make use of the specific geometry of the sentry turret. Calculations of the angle followed along with defining the size center box of the camera's field of view. The code will try to align the outbox of the person to the center of the camera in order to increase the chance of a successful hit on the target.

The servo was initialized and calibrated next. This was done by setting the servos to the starting center position and then moving them according to the detection function located inside of the DepthAI code. After that it receives the values from the filter function; these new values update the coordinates of the servo making it center the outbox of the person

A print function was added for testing and refining purposes, this print function would output the coordinates of the outerbox of the person and the current degree of

the servo.(Figure 1.2.4) Further on the code checks to see in the outerbox of the person is aligned with the center box of the camera and if not it will move the servo accordingly by measuring the coordinate distance from the outerbox and the center box. These values can either be -X.xx for the left side and +X.xx for the right side. The code will try to reduce the values to 0.00 which is the center of the camera. (Figure 1.2.5)

```
while True:
    if stop_threads:
        break

    if time.time() < next_t:
        time.sleep(next_t - time.time())
        next_t = next_t + 1.0/fps

    # Read the latest spatial coords, updates vars, gets filtered vals
    # This is mildly dangerous to read unprotected data like this, but
    # I had issues with using a queue.Queue due to fps sync. </shrug>
    xf = x.update(spatial_coords[0])
    yf = y.update(spatial_coords[1])
    zf = z.update(spatial_coords[2])

    # Just some logging stuff.
    if time.time() >= next_log_t:
        xd = f'{xf:.3f}' if xf is not None else '---'
        yd = f'{yf:.3f}' if yf is not None else '---'
        zd = f'{zf:.3f}' if zf is not None else '---'
        print(f'xf: {xd} // yf: {yd} // zf: {zd} // pan: {pan_angle:.1f} // tilt: {tilt_angle:.1f}')
        next_log_t = next_log_t + 1.0
```

Figure 1.2.4

The firing mechanism was the easiest part to implement. It was implemented into the servo function, more specifically in the part that checks for the alignment of the outerbox and the center box. Before implementing this once the two boxes were aligned the code would use the “*pass*” function but now it will also turn on the relay. This action would as well turn on the motor of the main weapon. Respectively when the boxes are not aligned the relay would be switched off thus powering down the main weapon.(Figure 1.2.5)

```
if xf is None:
    #GPIO.output(24, 0)
    #print("squeese jit")
    #time.sleep(0.5)
    pass
elif xf > center_thresh:
    pan_angle = unnorm_angle(pan.value) - angular_vel / fps
    #GPIO.output(24, 1)
elif xf < -center_thresh:
    pan_angle = unnorm_angle(pan.value) + angular_vel / fps
    #GPIO.output(24, 1)
```

Figure 1.2.5

This represents the software part of the project being presented in big lines only focusing on the necessary and critical components of the product. Inspiration was taken from a GitHub repository in order to complete this project.[3]

# Testing

The testing of the product was done during the whole development of the sentry turret in order to further improve it as the project moved forward; hence why the testing was separated into multiple parts.

## Servo test

The servo test was done by writing a simple code in order to test turn the servo from 0º to around 200º. This was done by using the python provided library GPIOzero and by setting the pulse width to 1/2500. Testing and operating the servos was very difficult due to the fact that there was no access to  a servo shield and this was a great impediment to the progress of the project. (Figure 2.1.1)

```
from gpiozero import Servo
from time import sleep

servo = Servo(25,min_pulse_width = 1/2400)

try:
        while True:
                servo.max()
                sleep(5)
                servo.min()
                sleep(5)
except KeyboardInterrupt:
        print("Program stopped")
```

Figure 2.1.1

Having to rely on a digitally generated PWM(Pulse Width Modulation) meant that the servos weren't getting a stable signal and by utilizing the pins of the Raspberry Pi4 meant that there was a possibility for a power surge to happen on the servo which would drain the voltage of the Raspberry Pi4 and force it to enter safe mode and automatically restart. This was a big impediment on the progress of the project as the servo was almost always active and moving.

The tests had to continue without a shield but after many attempts were successful by using an auxiliary 9V battery only implemented for testing purposes. The turret had the desired ~200º turn radius and no cables were hindering this process. This code represents only the testing stage of the servo and the code was modified during the Full test. A problem encountered during testing is that the servo, whilst being powered by the main vero board, would not be operational, refusing to turn entirely. Multiple tests were run using different pins on the Raspberry Pi4 and even changed the servo, assuming it was accidentally damaged during the previously run tests. This did not fix the problem even though the voltage given to the motor was the same as the servo. After carefully looking into it it was noticed that the Raspberry Pi4 needed to have the same ground as the board when sending the signal. This was fixed by connecting the Raspberry Pi4s ground pin to the ground line in the main vero board. This successfully allowed control of the servos at maximum efficiency.

Although the high voltage running through the servo might limit its half life it is suitable to use for this prototype as it will not be constantly running as the final product is intended to do. The test of the servo was a success, showing a high sturdiness and responsiveness even with the digitally generated PWM signal.

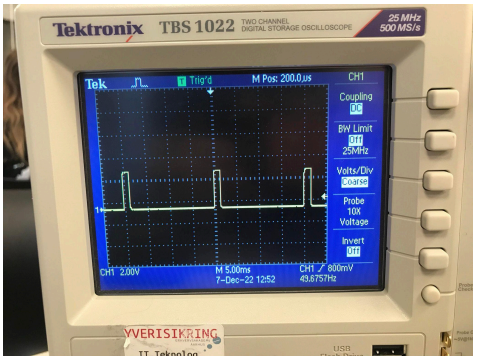



Figure 2.1.2

## Camera test

The testing of the camera was facilitated by the files provided by the Luxonis website and respectively using the provided GitHub repositories. All of the necessary files were downloaded to the Raspberry Pi4 and after installing the required libraries the camera was ready for testing. The testing of the camera went smoothly as the provided files already had set test codes for each part of the camera. Testing the face recognition, the coordinate calculator and the noise map. No modifications were done to the camera code during testing keeping the code stock.[4]

## Vero board test

The vero board had a rather lengthy test process. The main board was tested right after completing it by connecting it straight to the battery and checking the given voltage with a Voltmeter. The pins placed before the voltage regulator read ~9.74V and the pins after the voltage regulator read 5.07V. This concluded the main board test. (Figure 2.2.1)

The adjacent board was not tested before mounting and that was a big mistake. The relay was rated to only for 5V and 40 mAmps but the output of the main board was 10V with 10 Amps so that burned the relay and had to be switched with a 34.51 Finder PCB relay that is rather to 12V and 6 Amps which will hold for the testing period of the project. The new board was assembled and soldered using the same schematic as the old one, only replacing the relay and the twin resistor system for a single one with 390 Ohms resistance to match the resistance result. (Figure 1.1.9)


Figure 2.2.1

The board was tested by first connecting the battery to the board using 2 mm Radial connectors whilst checking the voltage with a Voltmeter. Once the required voltage was added and the relay clicked meaning that it was activated. Later on the motor of the main weapon was also connected using 2 mm Radial connectors to see if it would fire once the relay was engaged, which it did. The adjacent board was attached on top of the main board. And all of the necessary connections were soldered.

## Full test

The full test was done once the software part was concluded. The first impediment that appeared once the full test was running was that the vero board provided enough voltage and current to power on the Raspberry Pi4 but did not provide enough for the Raspberry to output an HDMI signal. This was of utmost importance as camera feedback was needed in order to rectify any possible issues that could be encountered. This also meant that the state of the Raspberry Pi4 was unknown and could not be operated confidently. For the purpose of the project the Raspberry Pi4 was reverted to using a power brick and a 220V standard outlet. An SSH connection has been made to the Raspberry Pi4 in order to control it remotely. In the future iterations of the project a more stable SSH connection so the Raspberry Pi4 could be monitored and configured remotely. (Figure 2.3.1)

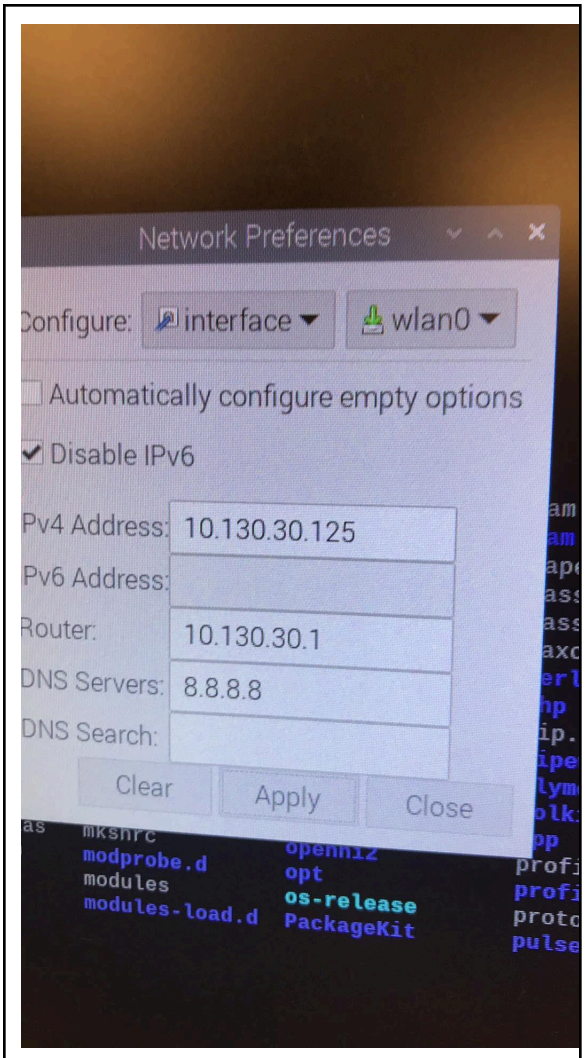



Figure 2.3.1

No other main or critical impediments or system failures were encountered during the rest of the test. The main impediment was created by the charge of the battery as it would only last around eleven to twelve hours rendering it obsolete for home defense. The provided nickel battery will be replaced with a higher capacity lithium-ion battery once the project passes the prototyping phase. The limited amount of life battery also meant that the testing and configuration of the product had to come to a halt in order for it to charge. It was also observed that the servo was undergoing a lot of “jitter” when the final code was running. This could be changed by either installing a servo shield on the Raspberry Pi4 or by changing the *max_pulse_width* and the frequency of the servo in the main code. An issue with the camera that was detected during its test while running the main code is that the camera would occasionally detect shadows or background elements as persons for one frame and would try to center those elements instead of the actual person.

This issue was simply fixed by positioning the product facing a flat white wall. The problem was encountered due to the system recognising persons using a noise map. This noise map can be influenced by uneven lighting or dark background elements. This will be fixed in the next production step on the product.

The whole system otherwise was successful, being able to detect the closest person, centering the outerbox of said person to the center of the camera and then firing the main weapon.

# Reflections

This project represented a challenge for my current skills and knowledge. During development I learned how to better manage myself and how to prioritize tasks so a completed project could be delivered. I did feel under equipped for this project and I did make a lot of simple mistakes that stopped the progress of the project entirely such as shorting the ground and the power in the main veroboard or not having a common ground for the raspberry, servo and vero board.

I definitely improved my problem solving skills by teaching myself how to think outside the box and come up with different approaches to solving a persistent issue. This project was a lot more complicated than any other project developed as part of the curriculum because I was in a one man team and nobody else was working on anything similar. This helped me develop myself and my mechanical and programming skills as I could not be aided by anybody and had to rely on myself to complete a task.

It was very exhausting and stressful but now that the project is complete I feel like I came a long way and I learned things that I didn't know before and I am pleased with the progress made on the project in the limited amount of time that I had at my disposal.

# Conclusion

The project was completed in the given amount of time. It was proven possible and with future developments the product could be used as an effective alternative of territorial defense, possibly cheaper to manufacture and maintain than training and deploying a fully trained agent.

The project was a success and was completed with minor drawbacks from the original concept. The development and implementation of this project pushed new boundaries of programming and mechanical knowledge.

The main purpose of the project was achieved and a functional product was delivered.

# Appendix

Proof of working product
https://youtu.be/YdBDih8OFEs

1 - Project description
https://eaaa.instructure.com/courses/15816/assignments/35283?module_item_id=496731
2 - OAK-D camera software
https://github.com/luxonis/depthai-python/tree/main/examples/SpatialDetection
3 - GitHub repository
https://github.com/etotheipi/creepy_face_oakd_clip/blob/main/spatial_mobilenet_MOD.py
4 - OAK-D test
https://github.com/luxonis/depthai-python/blob/main/examples/ColorCamera/rgb_video.py